Disclaimer: These slides can include material from different sources. I'll be happy to explicitly acknowledge a source if required. Contact me for requests.

Machine Learning in a Nutshell

15-288 Spring '26

Lecture 4:
Empirical vs. Generalization Errors,
Loss Functions

Teacher:
Gianni A. Di Caro

جامعة كارنيجي ميلون في قطر
Carnegie Mellon University Qatar

Recap: Our car example

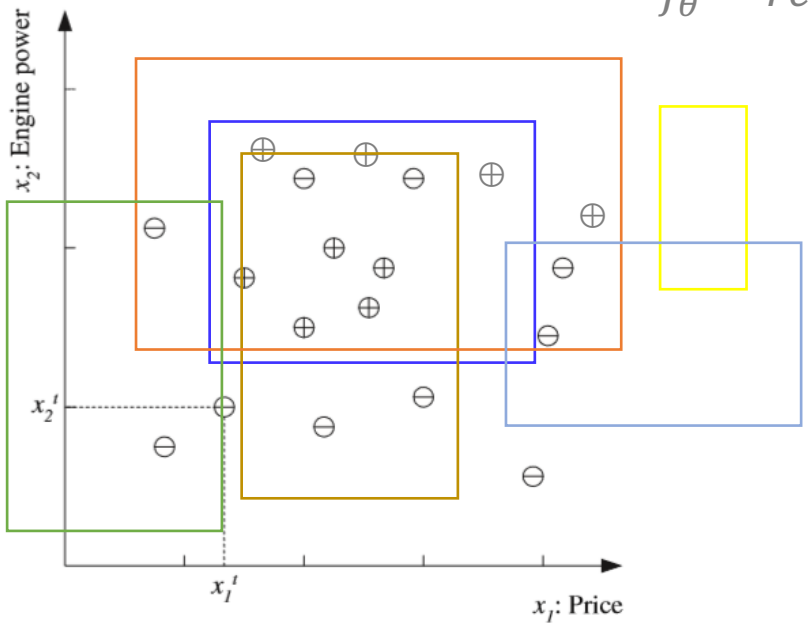
$+$ *Family car*
(positive example)

$-$ No family car
(negative example)

Learn the mapping $f: (price, power) \rightarrow Car\ class$

$f_\theta = rectangle\ (\theta_1, \theta_2, \theta_3, \theta_4)$

θ_1 = Min Price
θ_2 = Max Price
θ_3 = Min Power
θ_4 = Max Power



Hypothesis:
$f_\theta \in \mathcal{H}$ = set of axis-aligned rectangles

0/1 Loss:

$$\ell\left(f_\theta\left(\boldsymbol{x}^{(k)}\right), y^{(k)}\right) = \begin{cases} 1 & \text{if } f_\theta\left(\boldsymbol{x}^{(k)}\right) \neq y^{(k)} \\ 0 & \text{if } f_\theta\left(\boldsymbol{x}^{(k)}\right) = y^{(k)} \end{cases}$$

$$Total\ Loss = \sum_{i=1}^{19} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

E.g., $Total\ Loss$ $\square$ = 4

Recap: Train the model to minimize empirical error

Train & optimize the model:
Use the data in D to (*iteratively*, if necessary) set the values of the parameter vector $\boldsymbol{\theta}$ such that the total **loss** generated by f_θ on the data available in D is **minimized**

Optimization problem:

$$\min_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right) \quad \leftrightarrow \quad \boxed{\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)}$$

$$L_{emp}(\boldsymbol{\theta}; D) = \frac{1}{m} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

***Empirical* (mean) prediction error**

$$\boxed{\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} L_{emp}(\boldsymbol{\theta}; D)}$$

The Canonical Supervised ML Problem

1. Given a **dataset of training examples**, D, consisting of pairs of input features and target outputs $\{(\boldsymbol{x}^{(i)}, y^{(i)})\},\ i = 1, \dots, m$, and selected:
   - a **hypothesis function** f_θ,
   - a **loss function** ℓ,

   find (or approximate) **parameter values** $\boldsymbol{\theta}^*$ that **minimize the empirical error on** D (i.e., error on training data)

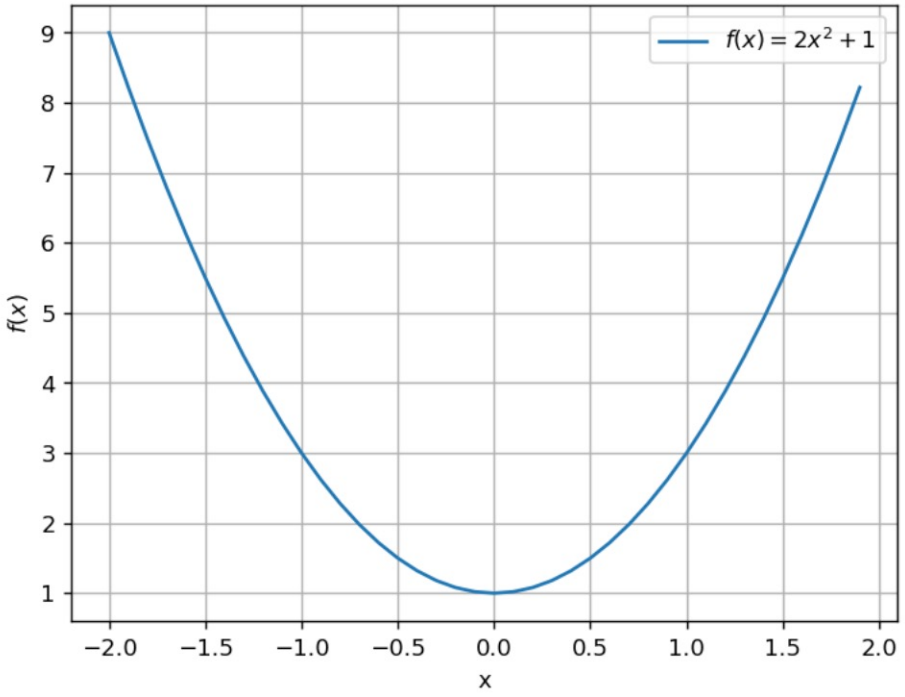
$$\boldsymbol{\theta}^* = \min_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

… Is this what are we *really* looking for? Is that all?

🤔

Do we understand the notation of optimization problems?

E.g., $f(x) = 2x^2 + 1$



$$z = \min_x f(x) \qquad \rightarrow z = 1$$

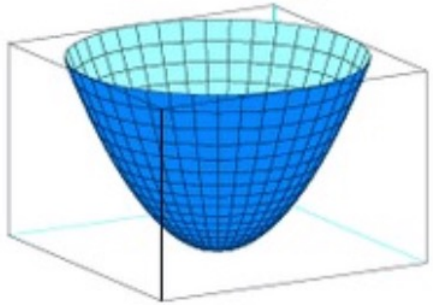
$$x^* = \arg\min_x f(x) \qquad \rightarrow x^* = 0$$

$$z \equiv f(x^*)$$

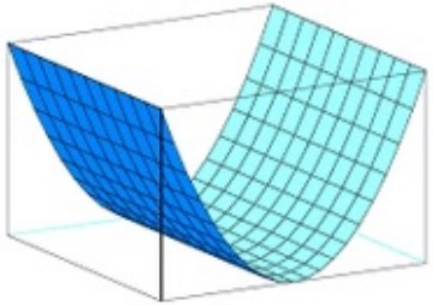
Multivariate optimization problems

$$x^* = (x_1^*, x_2^*, \dots, x_n^*)$$

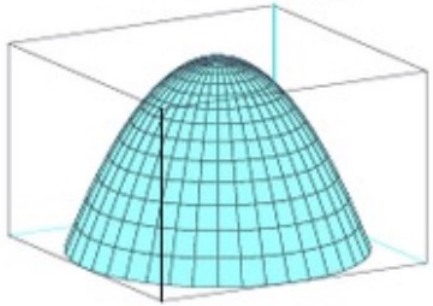
$f(x_1, x_2) = x_1^2 + x_2^2$



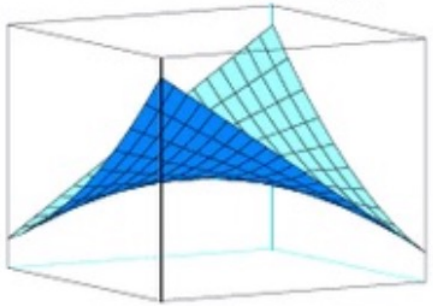
$f(x_1, x_2) = x_1^2$

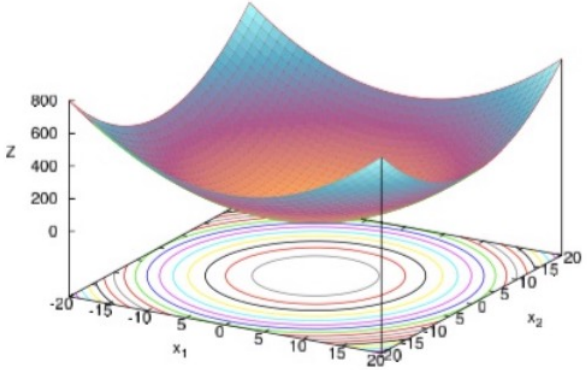


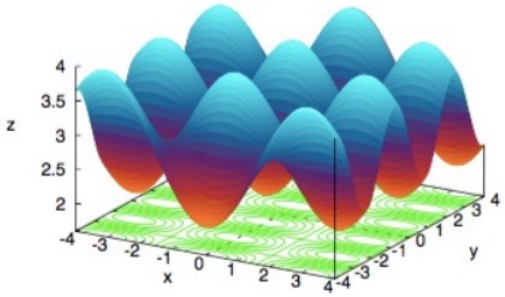
$f(x_1, x_2) = -x_1^2 - x_2^2$

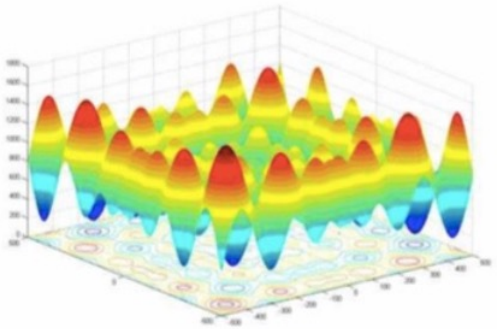


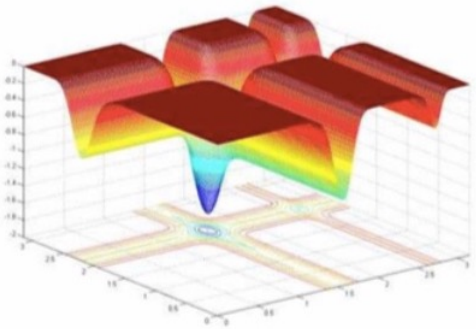
$f(x_1, x_2) = x_1 x_2 + 0.3x_2^2$











$$z = f(x_1^*, x_2^*, \dots, x_n^*)$$

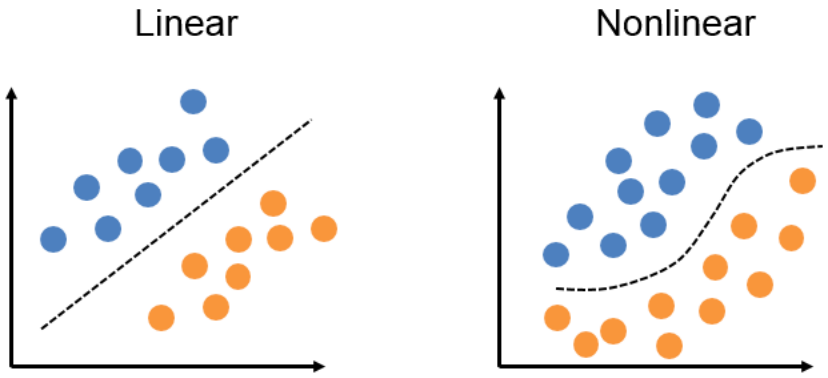
Do we understand the notation of optimization problems?

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

Example:

- 2-dimensional feature space → $\boldsymbol{x} = (x_1, x_2)$
- $f_\theta(\boldsymbol{x}) = f_\theta(x_1, x_2) = \theta_1 x + \theta_2 x_2 + \theta_0$ → **Linear separator** (a line in 2D)

Classification



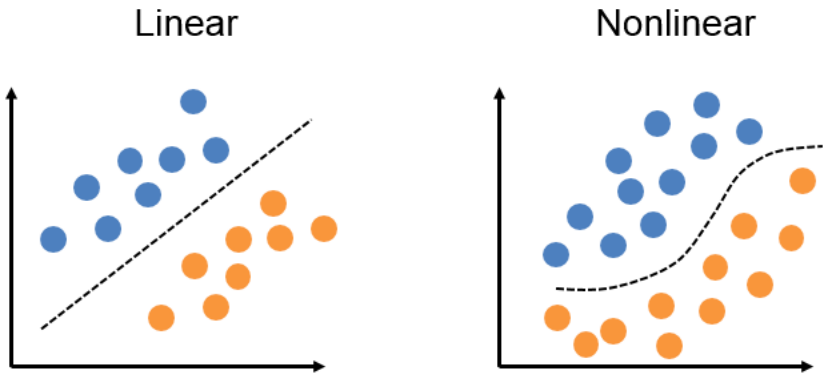
A linear example

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

Example:

- 2-dimensional feature space → $\boldsymbol{x} = (x_1, x_2)$
- $f_\theta(\boldsymbol{x}) = f_\theta(x_1, x_2) = \theta_1 x + \theta_2 x_2 + \theta_0$ → **Linear separator** (a line in 2D)

Classification



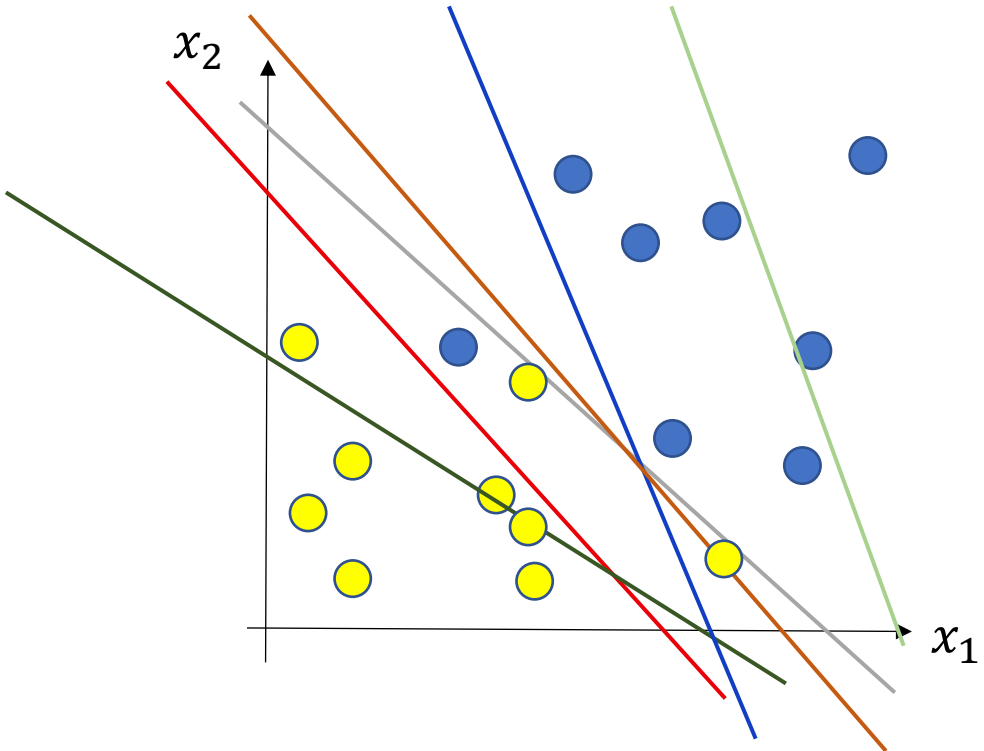
Linear models: linear combination of input features

- We *assume* that the target output $y = f_\theta(\boldsymbol{x})$, where y is a class or a continuous value, can be described by a weighted linear combination of the d values of the input features

$$f_\theta(\boldsymbol{x}) = \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d\, x_d + \theta_0$$

✓ → **Decision boundaries** are ***linear in the feature space***

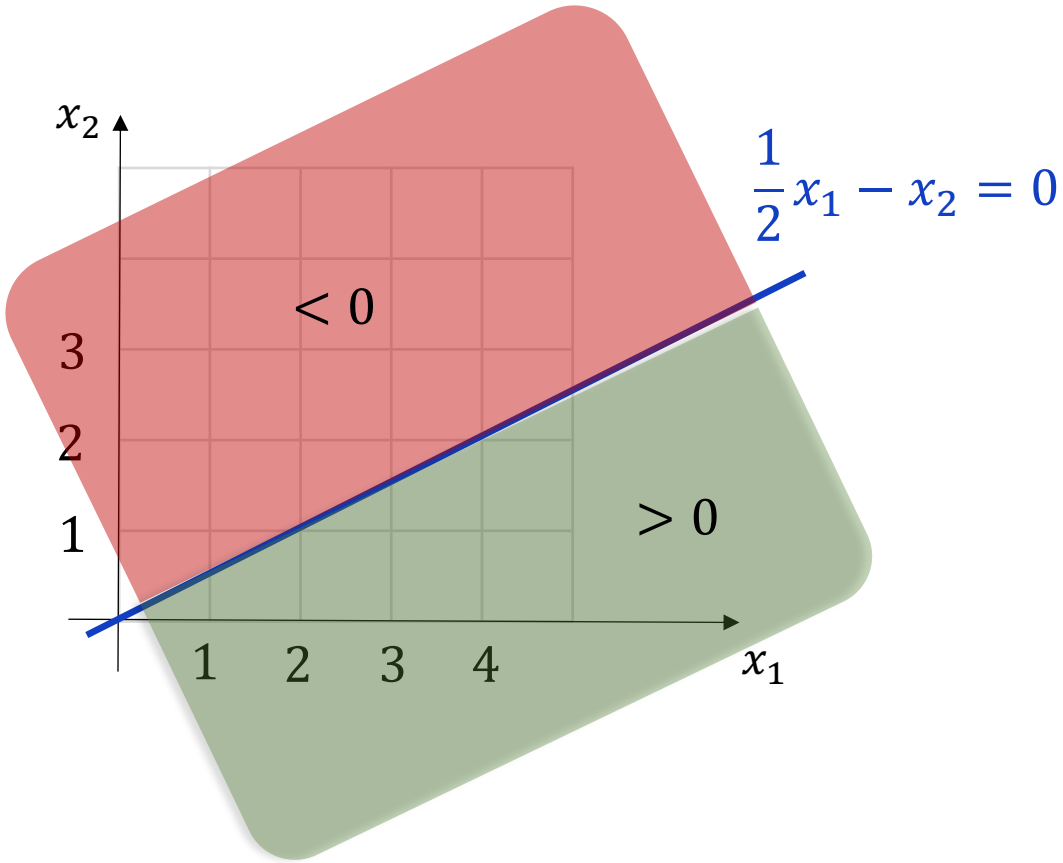
Find the *best* values for θ
(slope, intercept) using the training data



Linear models: basic geometry for binary case

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + w_0 = 0$$

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2} x_1 - x_2 = 0 \qquad \boldsymbol{w} = (0, \frac{1}{2}, -1)$$

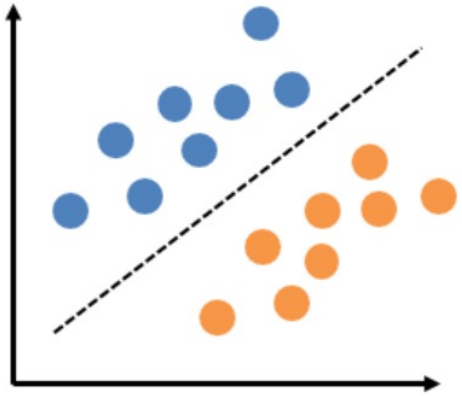


- ❑ $\boldsymbol{x} = (2, 1)$ is **on the line**
  - ✓ $f(\boldsymbol{x}) = 1 - 1 = 0$
- ❑ $\boldsymbol{x} = (3, 0.5)$ in **linear half-space** where $f_{\boldsymbol{w}}(\boldsymbol{x}) > 0$
  - ❖ $f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{3}{2} - \frac{1}{2} = 1 > 0$
- ❑ $\boldsymbol{x} = (1, 3)$ in **linear half-space** where $f_{\boldsymbol{w}}(\boldsymbol{x}) < 0$
  - ❖ $f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{1}{2} - 3 = -\frac{5}{2} < 0$

✓ The line of equation $f_{\boldsymbol{w}}(\boldsymbol{x})$ separates the region of the inputs $\boldsymbol{x}^+$, that determine a **positive value** of $f_{\boldsymbol{w}}(\boldsymbol{x})$, from the region of the inputs $\boldsymbol{x}^-$, that determine a **negative value** of $f_{\boldsymbol{w}}(\boldsymbol{x})$

Linear models for Classification (binary)

1. How do we use it to **make predictions**?
   - Which **classes** correspond to the values $1, -\frac{5}{2}, 99, \ldots$?

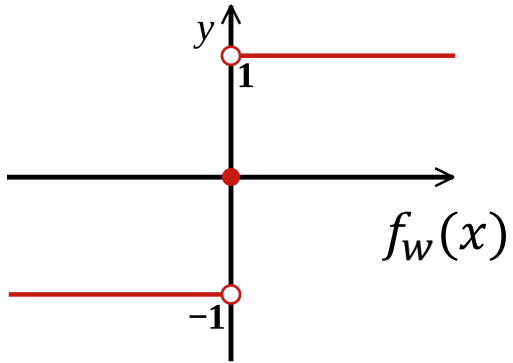


→ Need a *quantization / discretization* of the <u>continuous range</u> of values of $f_{\boldsymbol{w}}(\boldsymbol{x})$ in a set of <u>two values</u>

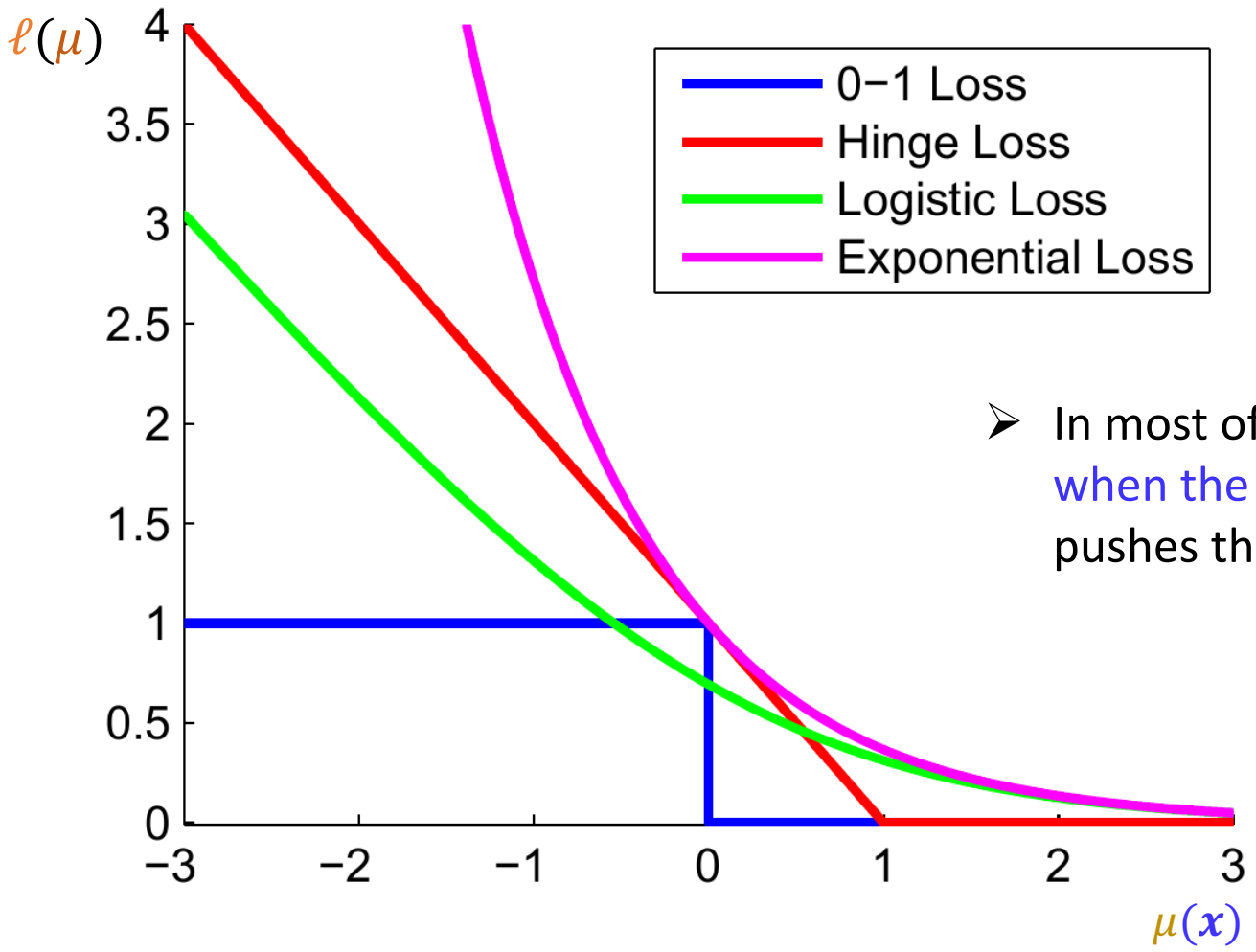
Note the procedures for the case of more than 2 classes are similar and can build on the binary case

➢ **Transformation** $\mathrm{sgn}(f_{\boldsymbol{w}}(\boldsymbol{x}))$ naturally outputs $\{-1, +1\}$

$$\mathrm{sgn}\big(f_{\boldsymbol{w}}(\boldsymbol{x})\big) : \boldsymbol{x} \rightarrow y \in \{-1, +1\}$$



Loss functions for classification



Losses (i.e., error corrections) of **different magnitude** in $(-\infty, +\infty)$ depending on <u>how bad (or good) the current prediction is!</u>

- In most of the cases, a loss is being generated even when the answer is basically correct, but the supervisor pushes the algorithm to learn an even better model!

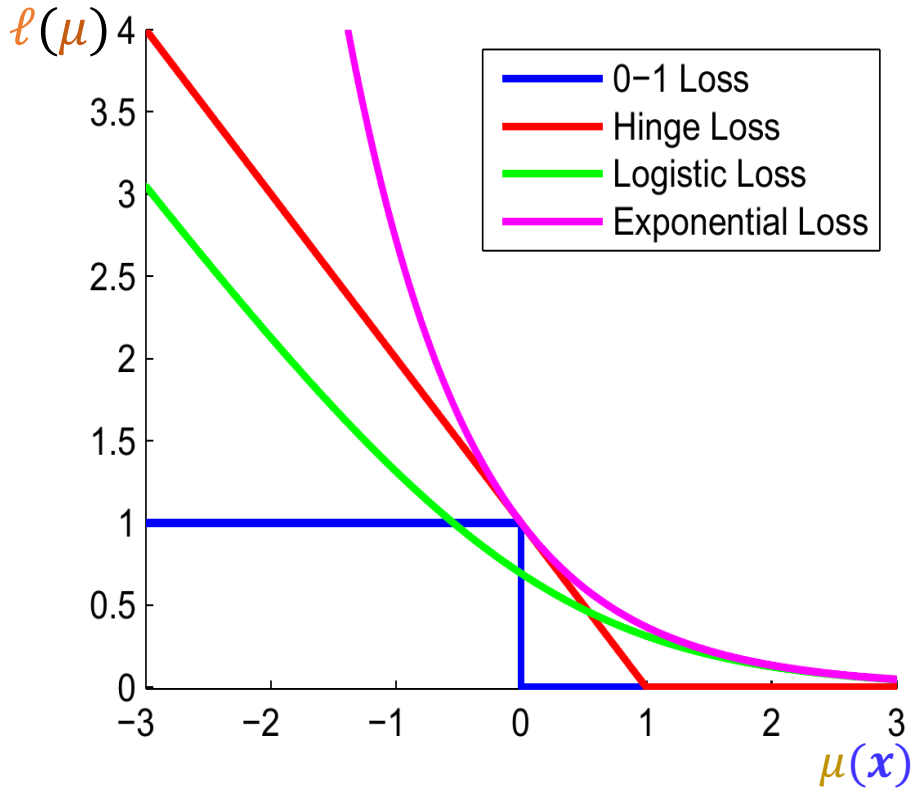
Margin: $\mu(\boldsymbol{x}) = f_{\boldsymbol{w}}(\boldsymbol{x}) \cdot y(\boldsymbol{x})$

✓ $\mu > 0 \rightarrow$ prediction agrees with label

○ $\mu < 0 \rightarrow$ prediction disagrees with label

For linear models: $\mu(\boldsymbol{x}) = (\boldsymbol{w}^T\boldsymbol{x} + b)y(\boldsymbol{x})$

Loss functions for classification



- 0-1: $\ell(\mu(\boldsymbol{x})) = \begin{cases} 1 & \text{if } \mu(\boldsymbol{x}) < 0 \\ 0 & \text{otherwise} \end{cases}$
- Hinge: $\ell(\mu(\boldsymbol{x})) = \max\{1 - \mu(\boldsymbol{x}), 0\}$
- Logistic: $\ell(\mu(\boldsymbol{x})) = \log\left(1 + e^{-\mu(\boldsymbol{x})}\right)$
- Exponential: $\ell(\mu(\boldsymbol{x})) = e^{-\mu(\boldsymbol{x})}$

How do we solve the optimization problem?

- **Given a loss function**, how do we **train / learn** a linear classifier?
  - How do we find the ***best*** linear decision boundary?
  - The ***best***?

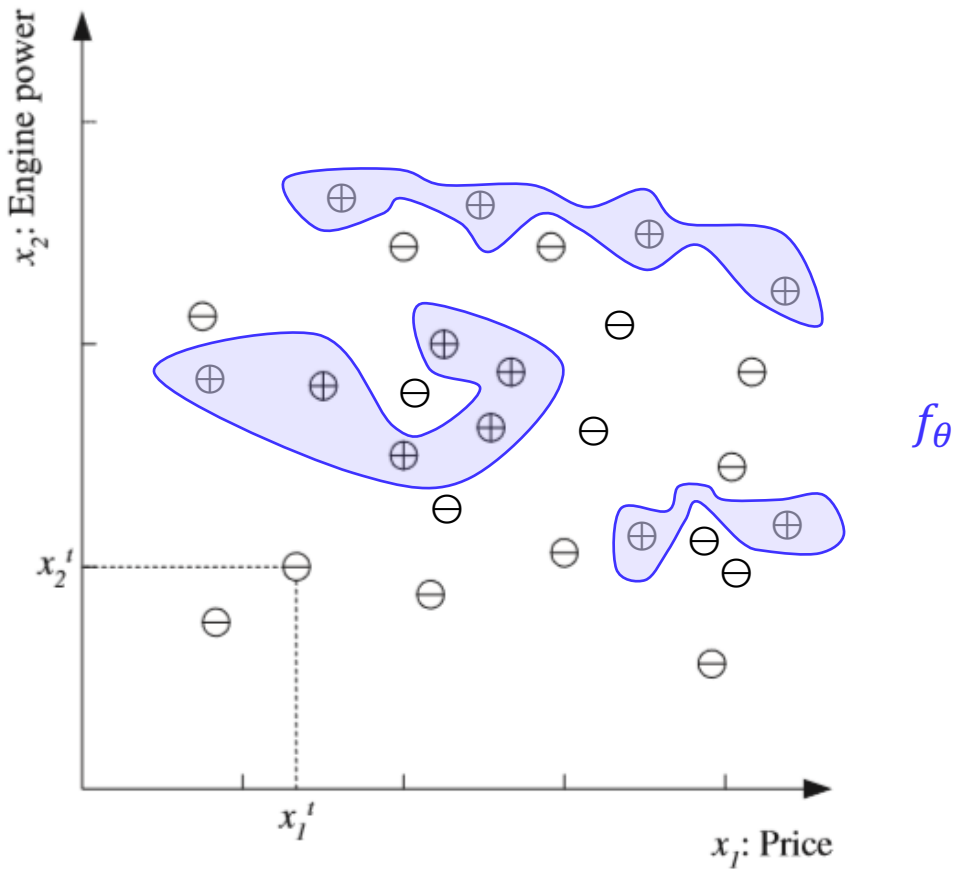
→ How do we solve the optimization problem?

$$\boldsymbol{w}^* = \min_{\boldsymbol{w}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(\boldsymbol{w}^T \cdot \boldsymbol{x}'^{(i)}, y^{(i)}\right)$$

Analytically is very hard to solve!!!

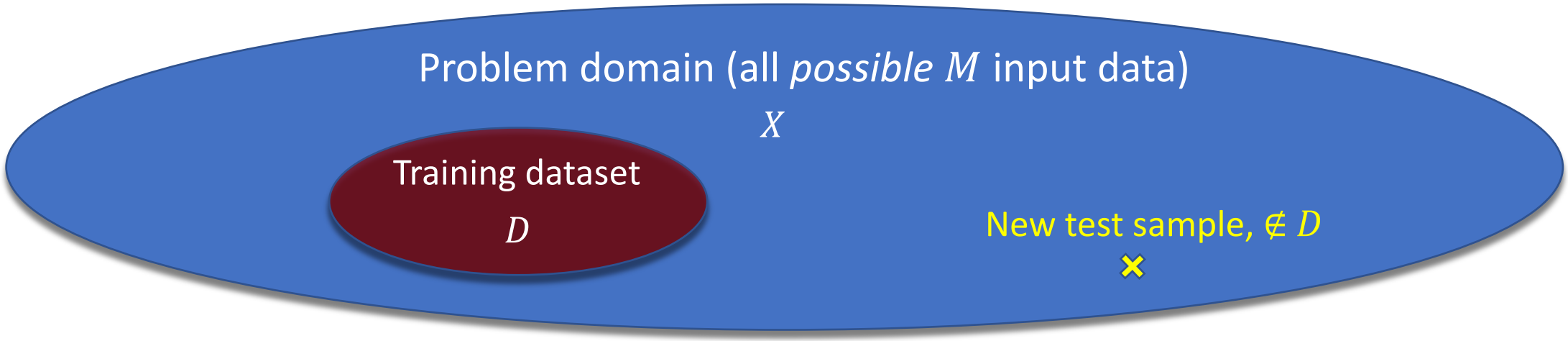
✓ Numeric / Iterative Approaches

Issues with *plain* empirical risk minimization



It's definitely minimizing the empirical loss! ... But would it be good on new, unseen data?

What are we looking for? → Minimize generalization error



True, unknown mapping is $f: X \rightarrow Y$

- **Goal of learning:**
  Minimize ***expected* loss** over *all* M pairs $(\boldsymbol{x}, y)$ in (X, Y) the learning machine *may see*

❖ **Fundamental problem in SL:** Use data in D to find the parameter values $\boldsymbol{\theta}$ (i.e., the hypothesis function f_θ) that minimize the expected ***prediction*** error on M (as measured by a *loss function* ℓ)

What are we looking for? → Minimize generalization error

- **Goal of learning:**
  Minimize ***expected* loss** over *all* M pairs $(\boldsymbol{x}, y)$ in (X, Y) the learning machine *may see*

→ Since $|M| \gg |D|$ what we really care is:

expected loss of prediction on *new (unseen) samples* $\boldsymbol{x}' \notin D$

Minimize:

✓ ***Generalization* error, *out-of-sample* error** when using the learned model f_θ

→ **Risk** of making errors using f_θ

Note on expected values

Probability theory note (you shall know this already)

- Let x be a *discrete* **random variable** that can take M values, $x_1, x_2, \cdots, x_M$, and $p(x_i)$ be the probability of the event that corresponds to x taking value x_i
  - E.g., in a fair dice throwing scenario, the showing face of the dice (the random variable) can take values $1,2,3,\cdots,6$ and the probability of each one of these possible events is 1/6

- The **expected value** of x, indicated by $E[x]$, is defined as:

$$E[x] = \sum_{i=1}^{M} x_i p(x_i)$$

Generalization error

- What matters: ***Expected* loss** over *all* M pairs $(\boldsymbol{x}, y)$ in (X, Y) the learning machine f_θ *may see*

✓ Given a loss function ℓ

- E.g., 0/1 count of classification errors: $\ell(f_\theta(x), y) = \begin{cases} +1 & \text{if } f_\theta(x) \neq y \\ 0 & \text{otherwise} \end{cases}$

(Expected) Generalization error: Mean loss/error over *all* possible data pairs for the problem

$$L_{gen} = \sum_{i=1}^{M} \ell\left(f_\theta(\boldsymbol{x}^{(i)}), y^{(i)}\right) P\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)$$

Prior probability distribution over the problem domain

$$L_{gen} = \sum_{i=1}^{M} \ell\left(f_\theta(\boldsymbol{x}^{(i)}), y^{(i)}\right) P\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)$$

What is this $P(X, Y)$?

SL Task: Learn the mapping $f: X \rightarrow Y$

- The mapping f defines all the M pairs $(\boldsymbol{x} \in X,\ y \in Y)$ that are *feasible* in the given **problem domain** $\wp$
- Moreover, in the problem domain $\wp$ each pair is associated to a probability (of appearance), $P(\boldsymbol{x}, y)$

Prior probability distribution over the problem domain

$f: bee \rightarrow weight$

$P(honey\ bee, 0.12) = 89\%$

$P(honey\ bee, 0.27) = 7\%$

(these are reasonable but random numbers!)

$(bee, 1\ kg)$ does not belong to the problem domain! (hopefully)

$\rightarrow P(bee, 1\ kg) = 0$

Prior probability distribution for the data of the problem

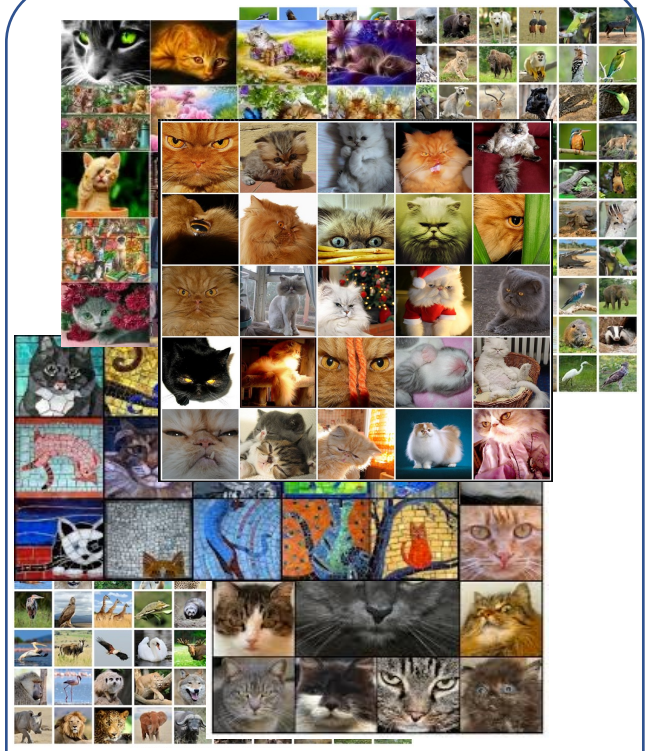
$$L_{gen} = \sum_{i=1}^{M} \ell\left(f_\theta(\boldsymbol{x}^{(i)}), y^{(i)}\right) P\left(\boldsymbol{x}^{(i)}, y^{(i)}\right)$$

P doesn't change, it is assumed **stationary**

$P(\boldsymbol{x}, y)$ is the **probability** of observing the specific pair $(\boldsymbol{x}, y)$ among all possible data samples in the problem domain

- Formally, to quantify L_{gen} we *would* need to define the **prior probability distribution** $P(X, Y)$ over the joint set (X, Y) of all *possible* example features X and targets Y !!!

Prior probability distribution for the data of the problem



M
images

- Task: classify an image as a cat or not a cat, over all possible M images accessible from the Internet

$$Y = \{+1\ (cat), -1\ (not\ cat)\}$$

- $\boldsymbol{x} \in X$ are numeric image features (e.g., pixel values)
- $P(\boldsymbol{x}, y), \boldsymbol{x} \in X, y \in Y$ is the joint probability that feature values $\boldsymbol{x}$ correspond to a cat image or not (depending on y)

➢ $P(\boldsymbol{x}, y)$ is the probability of observing the pair $(\boldsymbol{x}, y)$ among all M possible images in the problem domain

Generalization error, risk using the trained model

✓ *Ideal* goal:
Minimize prediction loss L_{gen} on classifying every possible image in M

- E.g., $\ell =$ count how many mistakes

❖ Fact to face 1: M is huge 😖

❖ Fact to face 2: $P(X, Y)$ is not known! 🙄

➢ Never, we will have all those images and will be able to know $P(X, Y)$

❖ We can only hope to gather a relatively small(er) sample set, D

How to assess the Generalization Error?

- The generalization error, which is the **true prediction error** associated to the learned hypothesis f_θ, can only be estimated not precisely computed
- How can we make a *sound* **estimate** of the *generalization error*? 🤔

How to assess the Expected Generalization Error?

Two classes of approaches:

- **Theoretical:** derive statistical bounds on the difference between *true error* and *expected empirical error* (PAC learning, VC dimension)

- ✓ **Empirical (Practical, Statistically sound)**
  - Using a selected loss function, compute the **expected empirical error** on training dataset as a ***local indicator*** of generalization performance, and use it to optimize the parameters θ
    - Use separate datasets and/or datasets with held-out data to validate/test learned model
    - Use the expected empirical error *on the validation sets* as a statistical estimate of the generalization error and use it to further optimize θ (**Cross-Validation**)

Validation set: A labeled dataset that we DON'T use for training but only for testing/validating a model

Empirical risk

- We can use the **expected empirical loss** L_{emp} computed on the set of the $m \ll M$ available examples in D as an **empirical estimate** of the generalization loss:

$$\boxed{L_{emp}(\boldsymbol{\theta}; D) = \frac{1}{m}\sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)}$$

L_{emp} is computed as the *sample average* of the loss over the training set

- L_{emp} is an *empirical approximation* of the **risk** of incurring in prediction **errors** when using the hypothesis f_θ for classifying samples from $P(\boldsymbol{X}, Y)$ that are not in the training set

Empirical risk minimization

- **Empirical risk minimization principle:** the learning algorithm should choose the hypothesis f_θ that *minimizes the empirical risk:*

$$L_{emp}(\boldsymbol{\theta}; D) = \frac{1}{m}\sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right) \quad \rightarrow \quad \boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} L_{emp}(\boldsymbol{\theta}; D)$$

... which can be further applied to training and validation sets to better minimize expected generalization loss

The Canonical Supervised ML Problem

1. Given a **dataset of training examples**, D, consisting of pairs of input features and target outputs $\{(\boldsymbol{x}^{(i)}, y^{(i)})\},\ i = 1, \ldots, m$, and selected:
   - a **hypothesis function** f_θ,
   - a **loss function** ℓ,

   find (or approximate) **parameter values** $\boldsymbol{\theta}^*$ that **minimize the empirical error on** D:

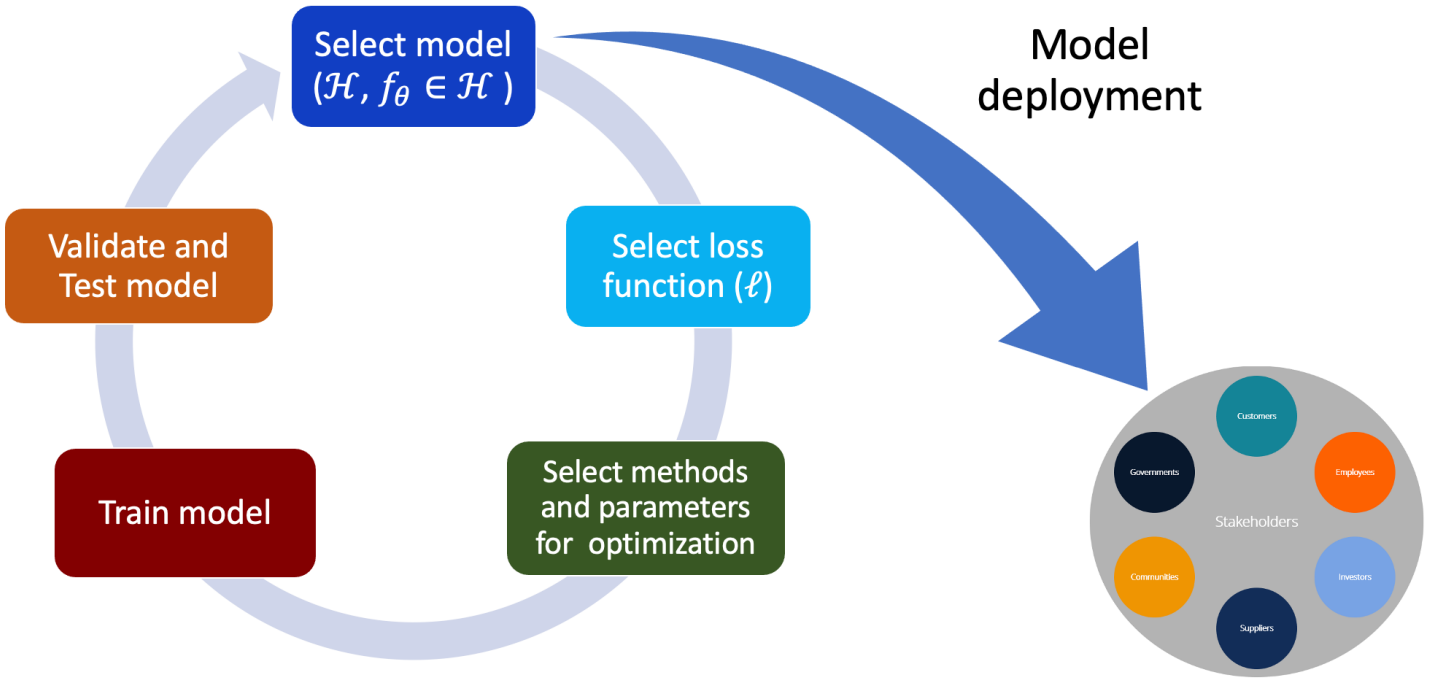
$$\boldsymbol{\theta}^* = \min_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$$

The Canonical Supervised ML Problem

2. If **validation sets** of examples, V, are made available (always possible in some way), find the parameter values θ^* that are the best ***tradeoff*** between:
   - **minimizing the empirical error on the training set**
   - and at the same time **minimize the empirical error on the validation sets**

The Canonical Supervised ML Problem

❖ The optimization process 1-2 can be iterated to select the model that provides the best tradeoff



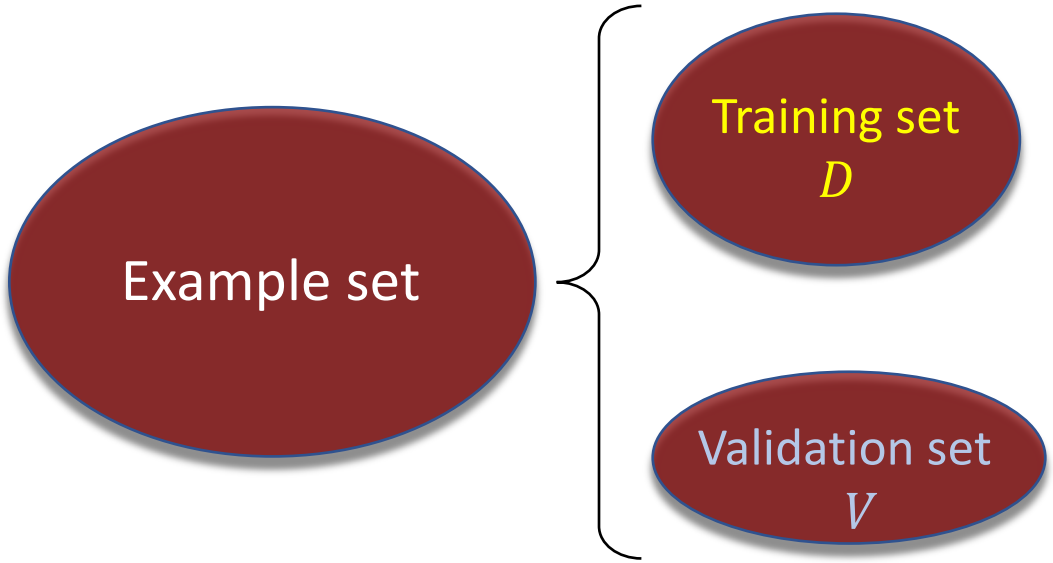
Empirical Loss on Validation sets as an estimator of Generalization Loss

- Split the available example set in:
  - ✓ **Training set**, D
  - ✓ **Validation set(s),** V (possibly, more than one set)

➢ Use minimization of L_{emp} on D to find optimized model parameters f_θ & get a *local* indication of L_{gen}

❖ Use the measure of **empirical loss** L_{emp} of f_θ on V as an estimate of **expected generalization loss,** L_{gen}

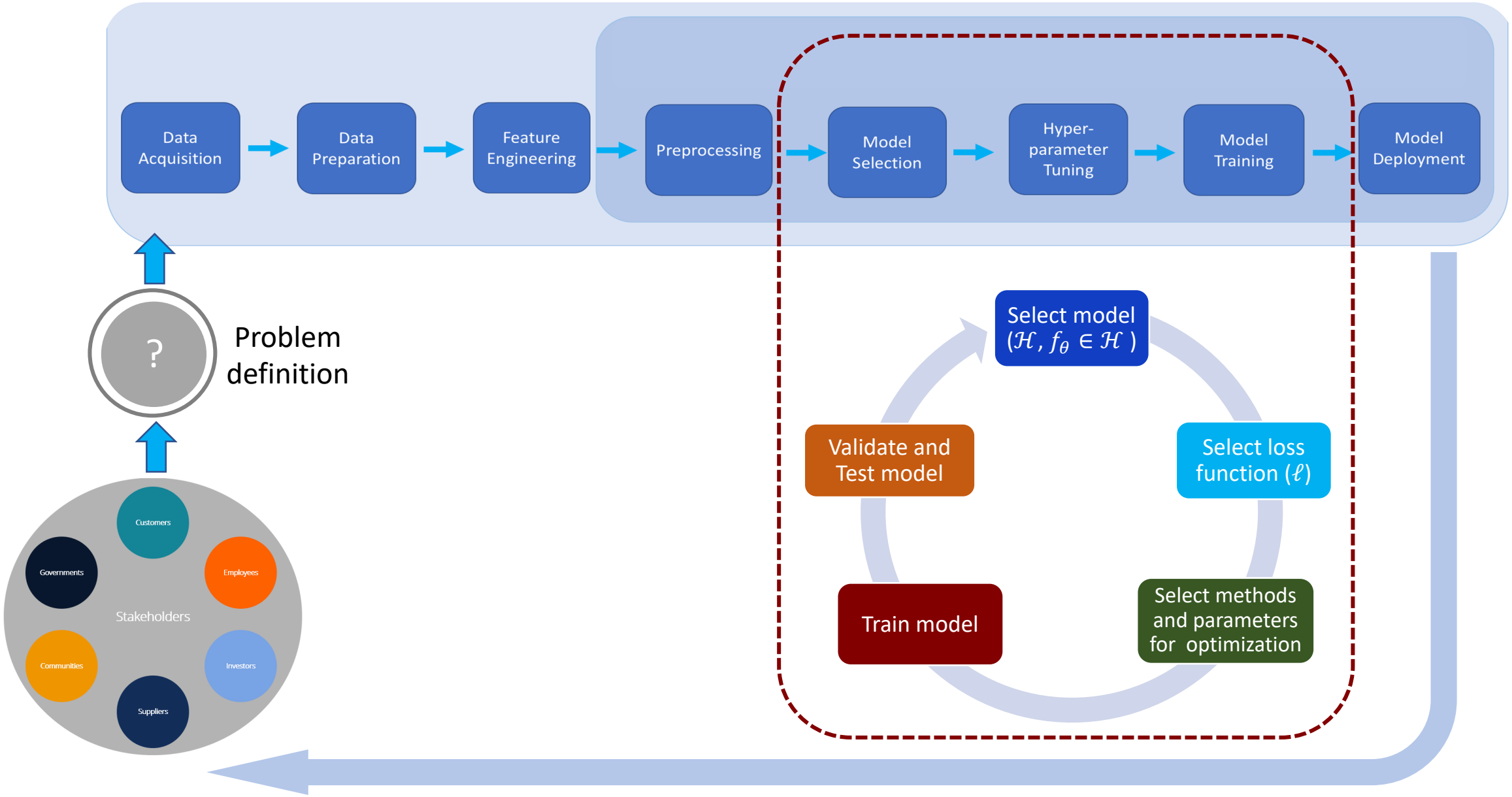
  ➢ Use it to further optimize model choice f_θ and its parameters θ (iteratively)



$$\min L_{emp}(D) \rightarrow f_{\theta^*}$$

$$L_{emp}(V) \approx L_{gen}(\mathrm{M})$$

Machine Learning Workflow / Lifecycle



ML Design for the Canonical problem

- Given Data & Problem definition, we need to specify:

  1. **The hypothesis class** $\mathcal{H}, f_\theta \in \mathcal{H}$ [Inductive bias, general hypothesis about the mapping]
  2. **The loss function** ℓ [Performance assessment]
  3. **The algorithm for solving the optimization problem** (often approximately) [Compute $\boldsymbol{\theta}$]
  4. **Procedure to create validation sets and to estimate generalization error** [Model selection]

How can generalization be *promoted*?

1. **Minimizing the empirical error**: $\min_{\theta} \bar{L}_{emp} \equiv \min_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)$

$\oplus$

2. Splitting the data in **training, testing, and validation sets** and finding the model with the best **tradeoff** of performance among the three sets (we'll see it later)

$\oplus$

✓ Acting on D: **gather large amounts of (representative) data to learn from,** which equals approximating the true data population X as much as possible, make $m \to M$

$\oplus$

✓ Acting on the loss function ℓ: add to it so-called ***regularization*** **terms** (we'll see it later)

$\oplus$

✓ Acting on the complexity / capacity of the hypothesized model f_θ: choose the hypothesis class $\mathcal{H}$ such that f_θ is *adequate* for the task, the dimensionality of the data, and the amount of the training data: **avoid overfitting**

Wrap-up 1

- Generalization is a key issue in ML
- Consistent hypotheses
- Loss functions
- Difference between error on training set vs. expected error on all possible samples for the task
- Empirical risk minimization principle and canonical ML problem
- Structure and challenges of a typical SL problem: hypothesis class, loss function, optimization
- Model selection and validation, overfitting, regularization are fundamental concepts related to generalization
- … we'll go through these concepts over and over during the course!

More on Loss functions

How do we evaluate *how* ***good*** a learned $f_\theta(\boldsymbol{x})$ is?

$$\boxed{L_{emp}(\boldsymbol{\theta}; D) = \frac{1}{m}\sum_{i=1}^{m} \ell\left(f_\theta\left(\boldsymbol{x}^{(i)}\right), y^{(i)}\right)}$$

- **Loss function**, ℓ**:** quantifies the prediction/classification *error* done by our hypothesis function $f_\theta(\boldsymbol{x})$ on **one training example**

ℓ(*predicted value*, *true value*)

→ Numeric quantification of the closeness between expected/true and predicted for the given example

Loss function: a core design choice

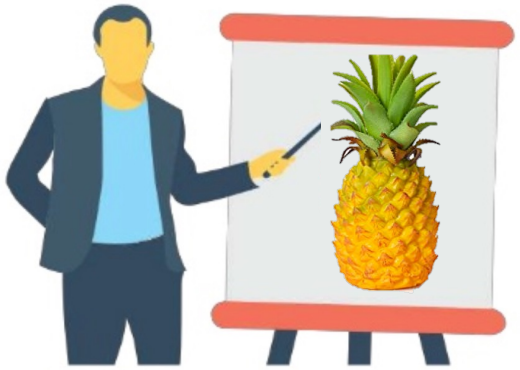
$$\ell\big(f_\theta(\boldsymbol{x}),\ \ y(\boldsymbol{x})\big) \to \mathbb{R}$$

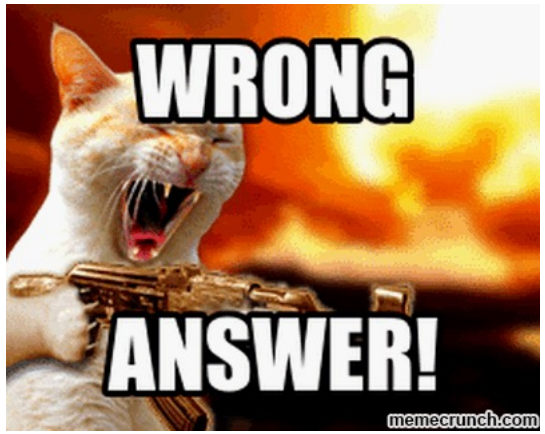
$$\ell\big(f_\theta(\boldsymbol{x}^{(k)}), y^{(k)}\big) = \begin{cases} 1 & \text{if } f_\theta\big(\boldsymbol{x}^{(k)}\big) \neq y^{(k)} \\ 0 & \text{if } f_\theta\big(\boldsymbol{x}^{(k)}\big) = y^{(k)} \end{cases}$$

0/1 Loss

- A loss function for the car example (classification)

Remember: Learning with a Teacher → Error correction





- ✓ **The model learned so far needs to be <u>modified</u>**
- ➢ *How?*
- ✓ **Accounting for the <u>error</u>**, quantified by the supervisor through the information carried by the label

Quantification of the error:
Loss function!

Examples of Loss functions for classification

❖ **Binary classification:** <u>**counting the errors**</u>

$$\ell(f_\theta(\boldsymbol{x}), y(\boldsymbol{x})) = \begin{cases} 1 & \text{if } f_\theta(\boldsymbol{x}) \neq y(\boldsymbol{x}) \\ 0 & \text{if } f_\theta(\boldsymbol{x}) = y(\boldsymbol{x}) \end{cases}$$

0/1 Loss

$$\sum_{\forall\, \boldsymbol{x} \in D} \ell(f_\theta(\boldsymbol{x}), y(\boldsymbol{x}))$$

Count **total** errors on dataset

- Usually written in more compact forms as an <u>indicator function</u>: $\ell(f(\boldsymbol{x}), y) = \mathbf{1}_{\{f(\boldsymbol{x}) \neq y\}}$

- Errors are treated as **symmetric** / **equivalent**:
  - Misclassifying a cat as a dog counts 1, the same as misclassifying a crocodile as a dog 😬
  - Misclassifying a sick person as a healthy person counts 1, the same as misclassifying a healthy person as a sick person 😏

Examples of Loss functions for classification

❖ **Binary classification: <u>weighted error counting</u>**

➢ Errors are treated as being potentially **asymmetric**

$$\ell(f_\theta(\boldsymbol{x}), y(\boldsymbol{x})) = \boldsymbol{A}_{\{f_\theta(\boldsymbol{x}) \neq y\}}, \quad \boldsymbol{A} = \begin{bmatrix} 0 & a \\ b & 0 \end{bmatrix}, \quad a \neq b$$

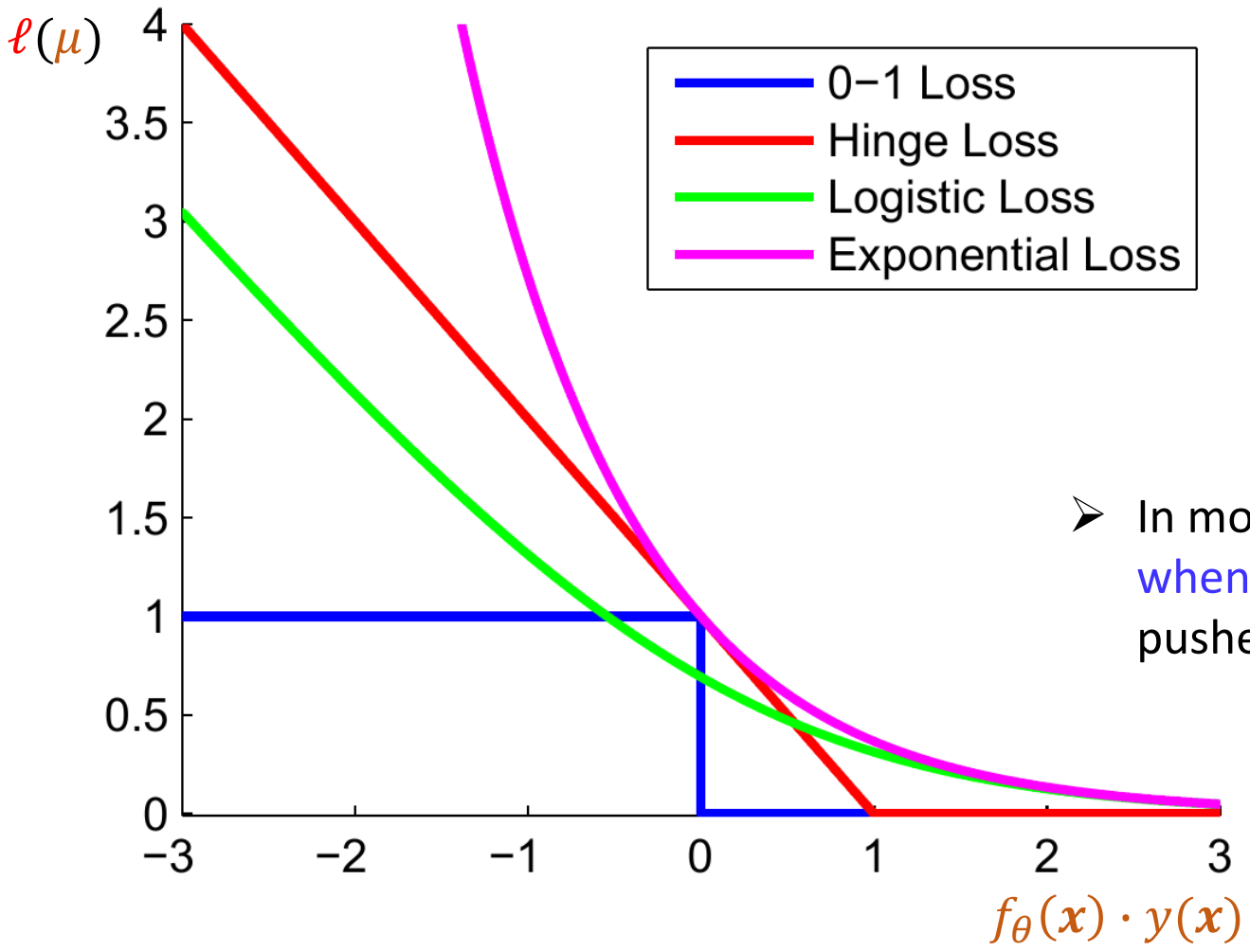
Asymmetric Loss

(not exactly 0/1 anymore)

- Two classes: $y \in \{c_1, c_2\}$
  - $\ell(f_\theta(\boldsymbol{x}), y) = 0$ if $f_\theta(\boldsymbol{x}) = y(\boldsymbol{x})$
  - $\ell(f_\theta(\boldsymbol{x}), y) = a$ if $f_\theta(\boldsymbol{x}) = c_1$ and $y(\boldsymbol{x}) = c_2$
  - $\ell(f_\theta(\boldsymbol{x}), y) = b$ if $f_\theta(\boldsymbol{x}) = c_2$ and $y(\boldsymbol{x}) = c_1$

- Misclassifying a sick person as a healthy person counts a, while misclassifying a healthy person as a sick person counts b, where $a > b$ 👍

More examples of Loss functions for classification



Losses (i.e., error corrections) of **different magnitude** in $(-\infty, +\infty)$ depending on <u>how bad (or good) the current prediction is!</u>

- In most of the cases, a loss is being generated even when the answer is basically correct, but the supervisor pushes the algorithm to learn an even better model!

Margin: $\mu(\boldsymbol{x}) = f_\theta(\boldsymbol{x}) \cdot y(\boldsymbol{x})$

- ✓ $\mu > 0 \rightarrow$ prediction agrees with label
- ○ $\mu < 0 \rightarrow$ prediction disagrees with label

Example of Loss function for regression: Square loss

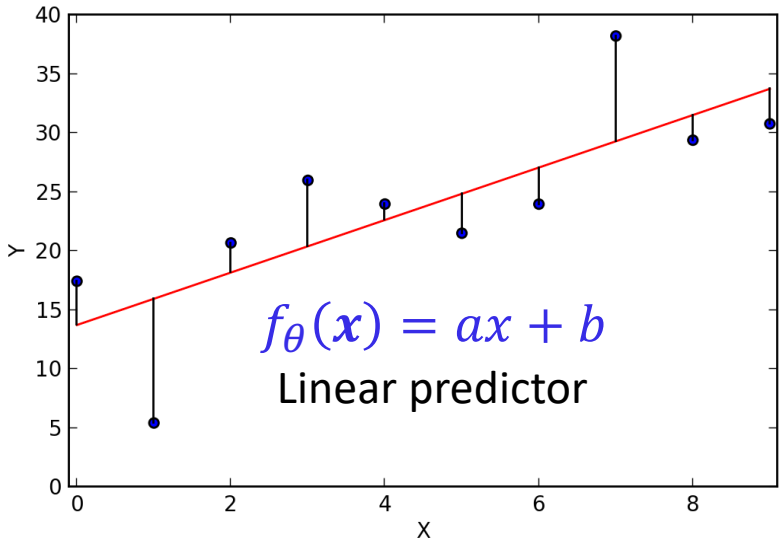
❖ **Regression: squared error**

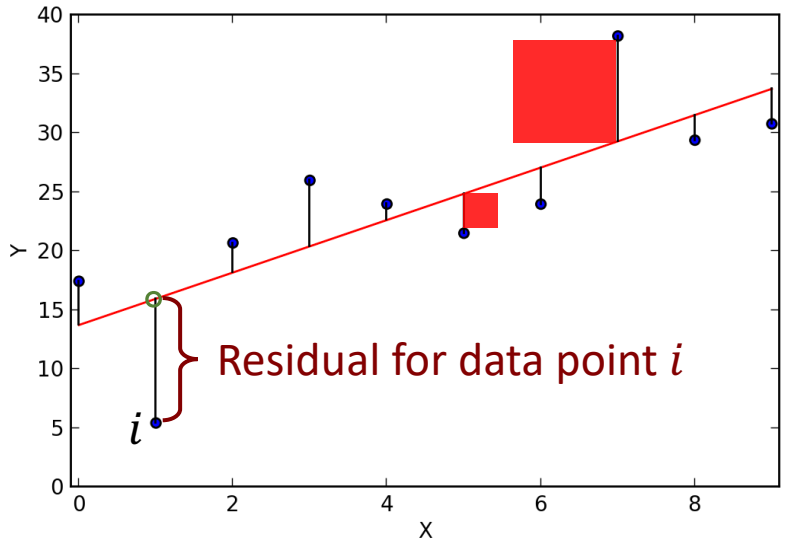
$$\ell(f_\theta(\boldsymbol{x}), y(\boldsymbol{x})) = (f_\theta(\boldsymbol{x}) - y(\boldsymbol{x}))^2$$

Square loss

✓ +ve and –ve errors are treated the same by squaring them

✓ Small errors (< 1) are reduced, large errors (> 1) are amplified by a quadratic growth

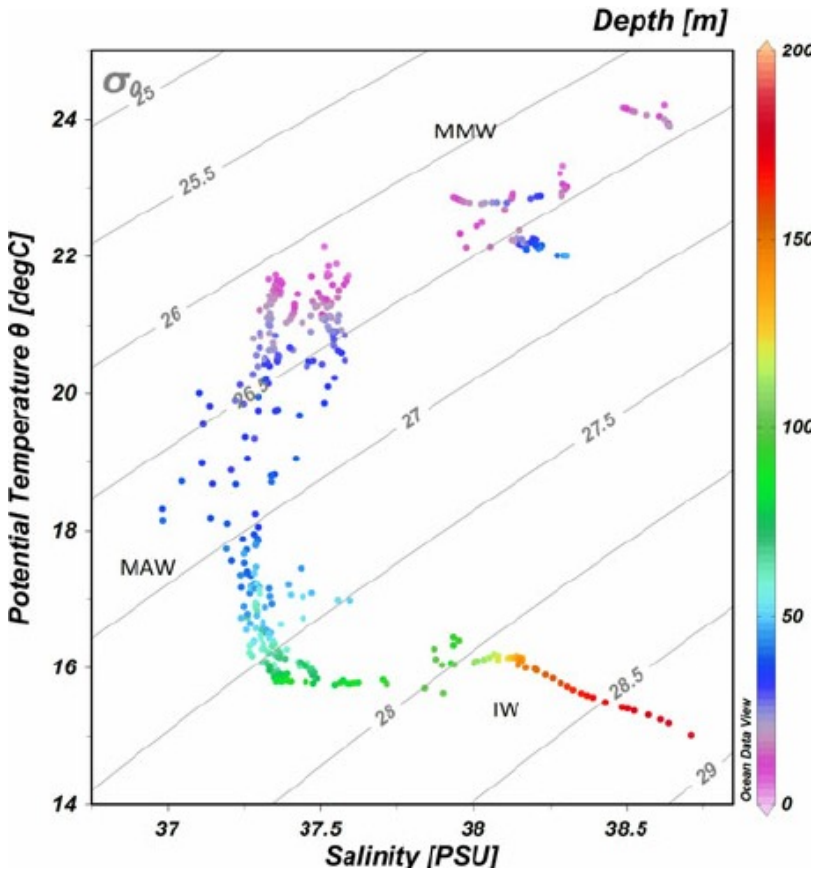




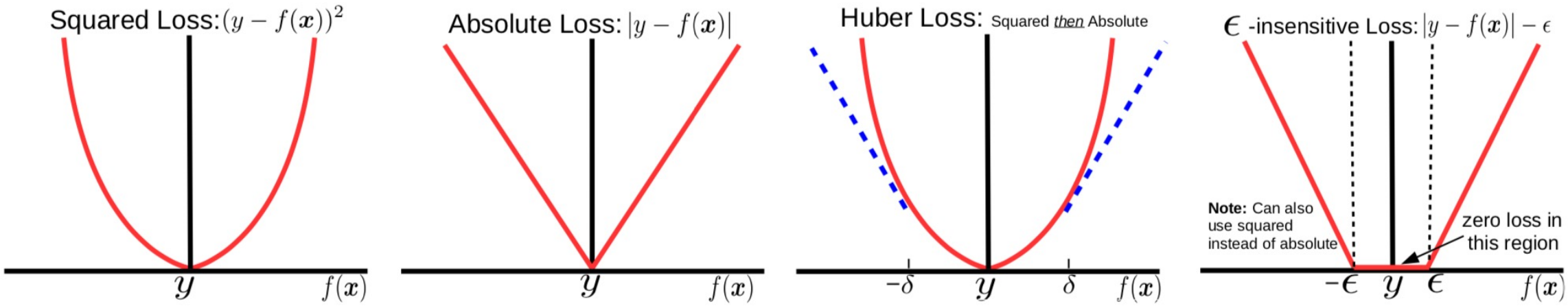
Example of Loss function for regression: Square loss

- $\boldsymbol{x} = (s, d)$ = Salinity and depth of ocean water
  - $\boldsymbol{x} = (37.5, 40)$
- y = Water temperature for a specific pair (s, d)
  - $y(\boldsymbol{x}) = 19.5$

- Errors with different learned models:
  - $f_{\theta_1}(\boldsymbol{x}) = 19.50 \rightarrow \ell = 0$
  - $f_{\theta_2}(\boldsymbol{x}) = 21.00 \rightarrow \ell = 2.25$
  - $f_{\theta_4}(\boldsymbol{x}) = 19.20 \rightarrow \ell = 0.09$
  - $f_{\theta_3}(\boldsymbol{x}) = 18.00 \rightarrow \ell = 2.25$

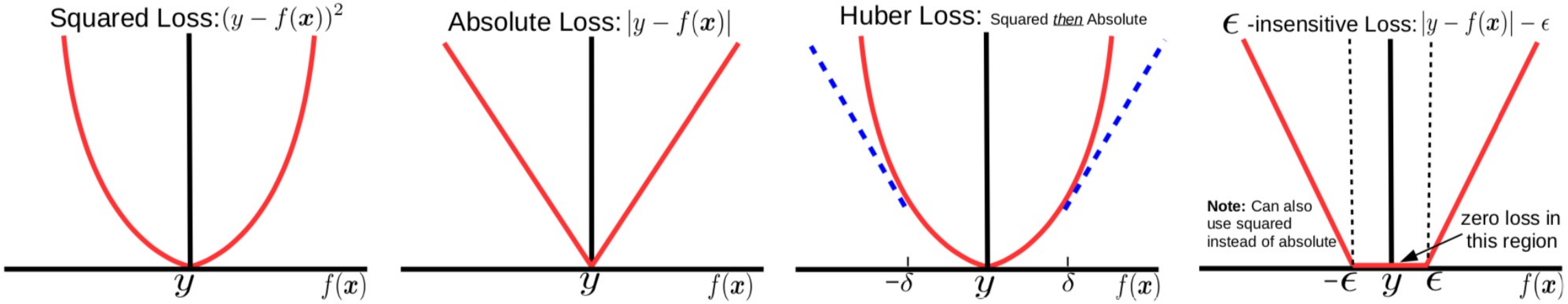


Commonly used loss functions for regression

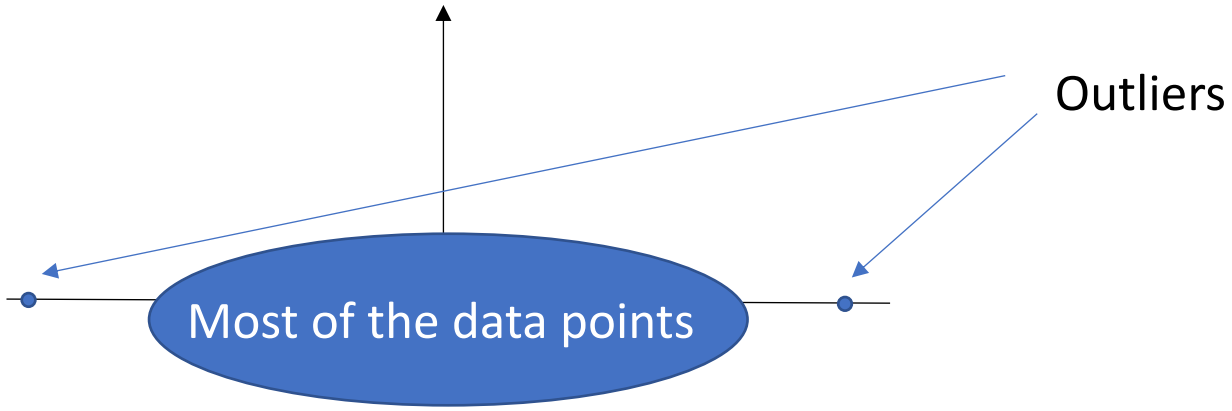


Issues with **derivatives** near 0
(derivatives are useful to find the minima of the function ...)

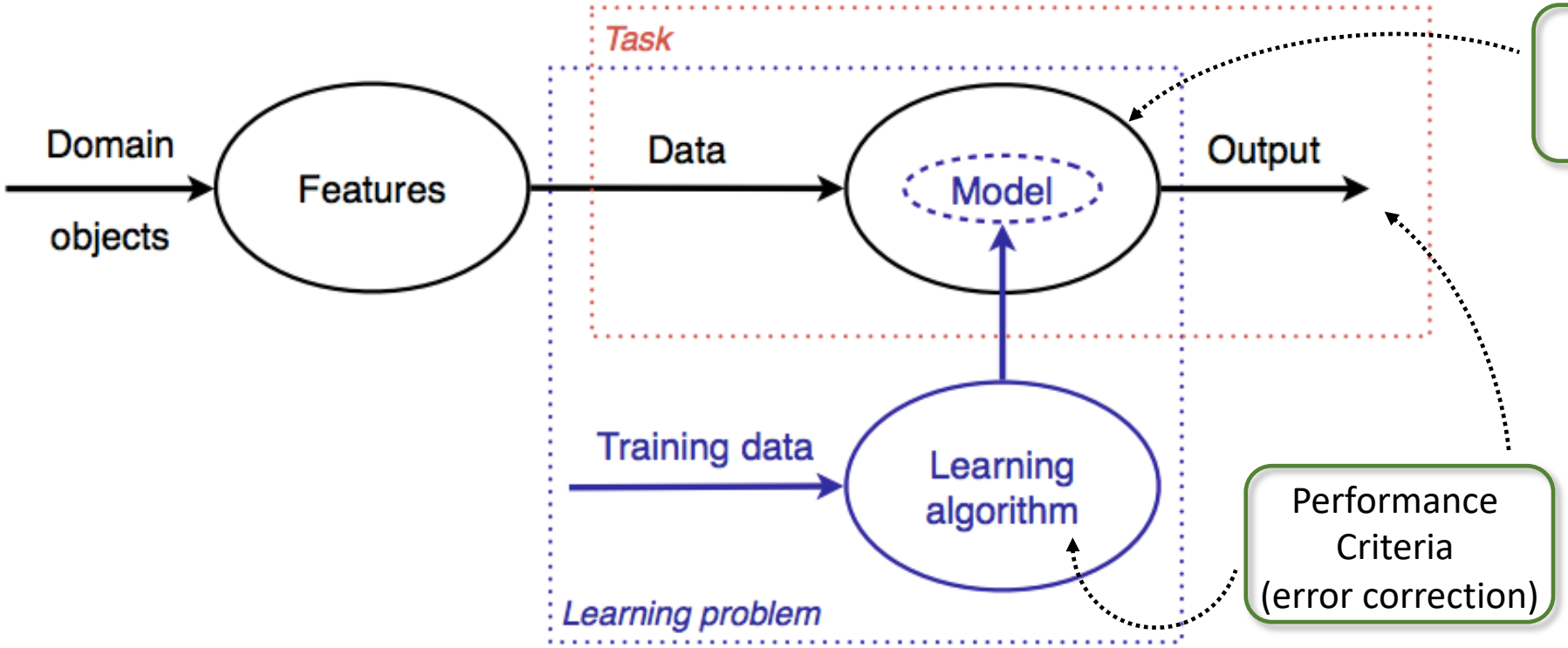
Quiz time



What will be the behavior of the different loss functions in presence of **outliers in dataset?**



Where we stand in General ML Scheme



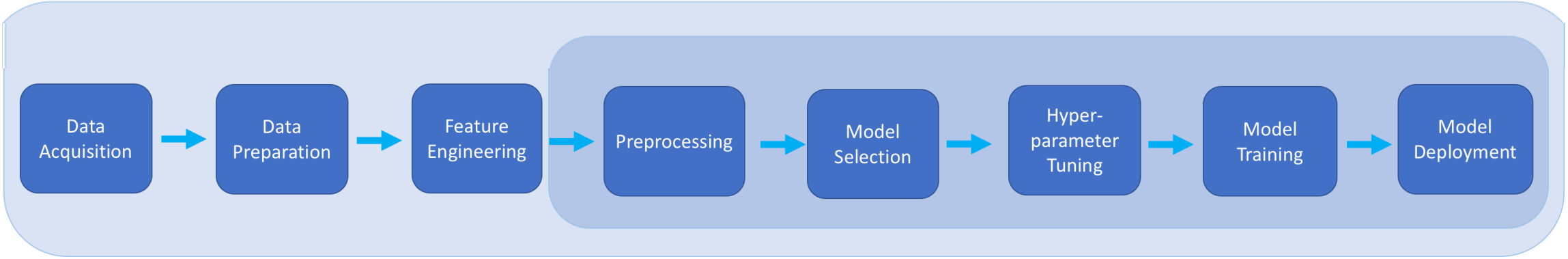
Hypotheses space

Hypothesis function

Performance
Criteria
(error correction)

ℓ(*predicted value*, *true value*)

Workflow of a (supervised) learning process



- **Data acquisition:** Get the training data, $D = \{(\boldsymbol{x}^{(i)}, y^{(i)})\}_{i=1}^{m}$
- **Exploratory Data Analysis (EDA)**: Explore the data (e.g., by visualization, by computing basic statistics)
- **Data wrangling**: … clean / scale / process the data / …
- **Features**: Identify and process relevant features, define the feature space X
- **Inductive bias:** Choose a hypothesis class $\mathcal{H}$, define a hypothesis function f_θ
  - The hypothesis function f_θ is the model of the ML task, it is defined by a vector of parameters $\boldsymbol{\theta} \in \mathbb{R}^n$
- **Loss function for model evaluation**: Select an appropriate loss function, ℓ, to evaluate the quality of f_θ
- **Train the model**: Use the data in D to iteratively set the values of the parameter vector $\boldsymbol{\theta}$ such that the loss produced by f_θ on the training set (and validation sets) is minimized
- …

Quiz time

1. The loss function is a mapping from the feature set to the label set. True or False?
2. The 0/1 loss function can be effectively used in regression problems, it would give a very precise assessment of the error and would drive good learning. True or False?
3. In classification, can I use a loss function which is not a binary one (i.e., it can give a 0.5 loss)?
4. When designing a loss function, its differentiability is an important property to take into account. True or False?

Wrap-up 2

- Workflow of a supervised learning problem scenario
- Structure and challenges of a typical SL problem: features, hypothesis class, loss function, optimization
- Choices mean inductive biases
- Loss function to score the effectiveness of learning
- Loss functions for classification and regression
- Generalization is a key issue in ML
- ... To be continued